

SKALA

4) AI 서비스의 고도화 생성형 AI 서비스 개발

4. Vector DB (코드 예제)

2026.10

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Today Checkpoint~!!!!!!!

이것만 기억해주세요~!!!!!!!

오늘의 핵심 ① 검색 품질은 '어떻게 자르느냐'가 결정

같은 모델, 청크 크기만 바꿨을 뿐인데

0.52 → **0.83**
100 토큰 500 토큰

Recall@5 = 정답이 상위 5개 검색 결과 안에
들어올 확률

너무 작으면 맥락이 끊기고,
너무 크면 의미가 흐려집니다.
모델을 바꾸기 전에 청크부터 점검하세요.

해법: **Parent-Child 전략** (두 마리 토끼)

찾을 때는 작게 · Child 200토큰
작고 정밀해서 관련 위치를 정확히 짚어냅니다



줄 때는 크게 · Parent 1000토큰
찾은 Child가 속한 Parent 전체를 LLM에 전달합니다

"Child로 찾고, Parent로 전달한다"

실무 기본값 300~500 토큰 + 10% 오버랩
→ 실습에서 청크 크기를 바꿔가며 Recall@5로 직접 확인

오늘의 핵심 ② 의미 검색의 맹점은 키워드 검색이 채웁니다.

검색어 **PostgreSQL 17.2 설치 오류 코드 42P01**

Dense 검색만 (의미 검색)

- 1위 DB 설치 개요 ✕
- 2위 SQL 기초 문법 ✕
- 3위 DB 성능 최적화 ✕

의미는 비슷하지만 버전·에러코드를 놓칩니다

Hybrid (Dense + BM25, RRF 결합)

- 1위 PG 17.2 릴리즈 노트 ✓
- 2위 42P01 해결 가이드 ✓
- 3위 PG 설치 오류 모음 ✓

정확한 키워드까지 잡아 정답이 상위로

Dense = 의미 이해

동의어·문맥 강함, 번호·코드 약함

BM25 = 단어 매칭

정확한 키워드, 새 단어도 즉시 처리

RRF = 순위로 합산

점수 대신 순위만 써서 안전하게 결합

판단 기준: Dense만으로 Recall@5 < 0.7 이거나 버전·코드·법령번호가 많다면 → Hybrid부터

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

8. 실습 – 벡터 검색 창고 만들기

- PDF 문서를 검색 가능한 형태로 바꿔서 저장합니다

실습 방향 — 오늘 우리가 만드는 것 & 큰 그림 먼저 보기

PDF 문서 → 질문하면 정확히 답하는 검색 시스템

- 실습 1: 문서를 컴퓨터가 이해할 수 있는 형태로 바꿔서 저장하는 '창고'를 만듭니다
- 실습 2: 그 창고에서 검색이 잘 안 되는 상황을 직접 겪어보고,
두 가지 방법(키워드+의미)을 합쳐서 개선합니다
- 핵심은 '왜 이 단계가 필요한지'를 이해하는 것 — 코드는 그 다음입니다

지금부터 한 걸음씩, 절대 서두르지 않고 갑니다

왜 Docker와 Qdrant를 미리 설치할까?

Qdrant = 우리가 만든 데이터를 보관할 '전용 참고 서버'

- 오늘 우리는 문서를 잘게 잘라 숫자(벡터)로 바꾼 뒤 어딘가에 저장해야 합니다.
- 그 저장 참고 역할을 하는 프로그램이 **Qdrant**이고,
Docker는 이 참고를 내 컴퓨터에 손쉽게 '설치 없이' 띄워주는 도구입니다.
- 즉, 오늘 하루 동안 잠깐 빌려 쓰는 '이동식 참고'라고 생각하면 됩니다.

체크리스트 — 순서대로 확인하세요

Qdrant 설치 (Docker, MacBook M1/M2/M3 + VSCode 기준)

- ☐ ① Docker Desktop 설치 확인 — 터미널에 `docker --version` 입력, 버전이 뜨면 OK
- ☐ ② Qdrant 실행
 - `docker run -p 6333:6333 -v $(pwd)/qdrant_storage:/qdrant/storage qdrant/qdrant`
- ☐ ③ 대시보드 접속 확인
 - 웹브라우저에서 `http://localhost:6333/dashboard` 접속, 화면이 뜨면 OK
- ☐ ④ Python 3.10 이상 확인 — `python --version`

환경 준비 — 문제 해결

이런 증상이 보이면	원인	해결 방법
docker: command not found	Docker Desktop 이 설치 안 됨	docker.com 에서 Docker Desktop 설치 후 터미널 재시작
포트 6333 이 이미 사용 중	이전에 띄운 Qdrant 가 아직 실행 중	docker ps 로 확인 → docker stop [이름] 후 재실행
대시보드 접속이 안 됨	Docker 컨테이너가 아직 켜지는 중	10초 정도 기다렸다가 새로고침

실습 1 전체 흐름 미리보기

6단계로 진행합니다

- ① **PDF 로딩** — 문서에서 순수 텍스트만 꺼내기
- ② **칭킹** — 꺼낸 텍스트를 적당한 크기로 자르기
- ③ **임베딩** — 잘린 텍스트를 숫자(벡터)로 바꾸기
- ④ **FAISS** — 벡터를 임시로 저장하고 검색해보기
- ⑤ **Qdrant** — 진짜 서버에 제대로 저장하기
- ⑥ **검색 비교 + 필터** — 두 저장소를 비교하고, 조건 검색 해보기

문서: **SPRi AI Brief** 등 5개 실제 보고서 사용 — 오늘 실습 자료 폴더 참고

오늘 실습에 쓸 문서 세트 — AI·반도체 실제 보고서

No	문서	발행처
1	SPRi AI Brief 2025~2026 6월	소프트웨어정책연구소
2	ETRI 인공지능 실행전략	한국전자통신연구원
3	삼일PwC 반도체 산업 전망 보고서	삼일PwC
4	AI 반도체 강국도약 가이드라인	관계부처 합동
5	AI 반도체 기술 및 산업 동향	산업 동향 보고서

실제 보고서로 실습하기 때문에, 여러 문서를 종합해야 답이 나오는 질문도 경험합니다

실습 방향 — 오늘 만드는 것

실습 1 — "벡터 DB가 왜 필요한지 체감"

- **PDF 로딩 → Chunking → BGE-M3 임베딩 → FAISS 구축 → Qdrant 구축 → 검색 비교 → 필터 검색**

실습 2 — "Dense 한계를 직접 확인하고 개선": BM25 → RRF → Reranker → RAGAS 검증

실습 1 흐름 — PDF → FAISS → Qdrant

1 1~2. PDF 로딩 · 청킹

PyMuPDF + RecursiveCharacterTextSplitter

2 3. 임베딩 생성

BGE-M3로 벡터 변환

3 4. FAISS 인덱스 구축

HNSW 인덱스 + 검색 테스트

4 5~6. Qdrant 구축

Collection 생성 + Upsert

5 7~8. 검색 비교 · 필터 검색

FAISS vs Qdrant, 메타데이터 필터

PDF 로딩 — 왜 필요한가?

개념

PDF는 '사람이 보기 좋은 종이', 컴퓨터에겐 '글자만 쓱 빼낸 메모장'이 필요합니다

- **PDF** 안에는 글자뿐 아니라 디자인, 이미지, 표 배치 정보까지 잔뜩 들어있습니다.
- 컴퓨터가 검색하려면 이런 꾸밈은 다 떼어내고 '순수 텍스트'만 있으면 됩니다.
- **PyMuPDF(fitz)**라는 도구가 **PDF** 파일을 열어서 페이지별로 글자만 쓱쓱 꺼내줍니다.

RAGAS(ragas.io) — RAG 전용 평가 프레임워크, 오늘 종합실습에서 직접 사용합니다

Step 1. PDF 로딩 — 문제 해결

이런 증상이 보이면	원인	해결 방법
총 0페이지 로딩 완료	스캔된 이미지 PDF (글자가 그림임)	다른 문서로 실습 진행
FileNotFoundException	파일 경로가 틀림 / 실행 폴더가 다름	터미널에 pwd 입력해 현재 위치 확인 후 절대경로 사용
글자가 깨져 보임 (□□□)	폰트·인코딩 문제 (드물게 발생)	다른 PDF 리더로 원본이 정상인지 먼저 확인

Chunking — 왜 통째로 쓰지 않을까?

개념

책 한 권을 다 외우게 하지 말고, 소제목 단위로 찢어서 필요한 페이지만 찾게 하자

- 100페이지짜리 문서를 AI에게 주면 비용도 크고, 정작 필요한 한 문단을 놓치기 쉽습니다.
- 그래서 문서를 500자 정도의 작은 조각(청크)으로 미리 잘라둡니다.
- 조각 사이를 살짝 겹치게(오버랩) 잘라서, 경계에서 문장이 뚝 끊기지 않게 합니다.

Step 2. Chunking — 문제 해결

이런 증상이 보이면	원인	해결 방법
청크 수가 너무 적음 (페이지 수와 비슷)	<code>chunk_size</code> 가 원본보다 훨씬 큼	<code>chunk_size</code> 를 500 이하로 줄이기
문장이 중간에서 뚝 끊김	<code>overlap</code> 이 너무 작거나 0	<code>chunk_overlap</code> 을 size의 10~20%로 설정 (50~100)
<code>IndexError</code> / <code>KeyError</code>	빈 텍스트에서 <code>split_text</code> 호출	Step 1에서 <code>text.strip()</code> 필터가 잘 되어있는지 확인

임베딩 — 글자를 숫자로 바꾸는 번역기

개념

컴퓨터는 '의미'를 모릅니다. 대신 '숫자 사이의 거리'는 계산할 수 있습니다

- BGE-M3라는 임베딩 모델에 문장을 넣으면,
그 문장의 의미를 담은 숫자 목록(벡터)을 돌려줍니다.
- 비슷한 의미의 문장은 비슷한 숫자 목록을 갖게 되고,
이 '숫자 사이의 거리'로 유사도를 계산합니다.
- BGE-M3는 무료로 다운로드해 내 컴퓨터에서 바로 돌릴 수 있고, 한국어 성능도 우수합니다.

Step 3. 임베딩 — 문제 해결

이런 증상이 보이면	원인	해결 방법
모델 다운로드가 오래 걸림	최초 실행 시 1회만 발생하는 다운로드	몇 분 정도 소요될 수 있음, 인터넷 연결만 확인하고 대기
CUDA out of memory	GPU 메모리 부족 (그래픽카드 없어도 발생 가능)	use_fp16=True 유지, batch_size를 16 → 8로 축소
속도가 너무 느림 (CPU만 있는 경우)	GPU 없이 CPU로 연산 중	정상 (청크 수백 개면 1~2분 정도) 오늘은 이 속도로 충분
shape가 (N, 1024)이 아님	return_dense=True를 빠뜨림	output['dense_vecs']를 사용했는지 확인

FAISS — 임시 검색 창고

개념

메모리 위에 잠깐 펼쳐놓고 빠르게 찾아보는 '작업대'

- FAISS는 벡터들을 컴퓨터 메모리에 올려두고,
그 안에서 아주 빠르게 유사한 것을 찾아주는 라이브러리
- 프로그램을 끄면 사라지는 임시 저장소라서, '일단 잘 되는지 빠르게 확인'할 때 사용
- HNSW는 앞에서 배운 '고속도로 구조' — 도시(대표 벡터)부터 찾아 들어가는 방식

Step 4. FAISS — 문제 해결

이런 증상이 보이면	원인	해결 방법
AssertionError (dimension)	인덱스 차원과 벡터 차원이 다름	IndexHNSWFlat(1024, 32) 에서 1024 가 맞는지 확인
검색 결과가 전혀 관련 없음	질문 벡터를 다른 방식으로 만듦	model.encode() 를 질문에도 동일하게 사용했는지 확인
TypeError: float64	FAISS 는 float32 만 지원	.astype('float32') 를 빠뜨리지 않았는지 확인

Qdrant — 진짜 창고에 제대로 저장하기

개념

FAISS가 '임시 작업대'라면, Qdrant는 '문서·태그까지 기록하는 정식 창고'

- FAISS는 벡터만 저장할 뿐, '이 벡터가 몇 페이지였는지' 같은 부가정보는 저장 못 합니다.
- Qdrant는 벡터 옆에 페이지 번호, 카테고리 같은 '태그(메타데이터)'까지 같이 저장해줍니다.
- 그래서 나중에 '5페이지 이후 문서에서만 찾아줘' 같은 조건 검색이 가능해집니다.

Step 5~6. Qdrant — 문제 해결

이런 증상이 보이면	원인	해결 방법
ConnectionError	Docker 컨테이너가 꺼짐	터미널에서 docker ps 로 확인 → 꺼져있으면 Step 0 의 docker run 다시 실행
Collection already exists	같은 이름으로 이미 만든 적 있음	client.delete_collection('rag_docs') 실행 후 재생성
Vector dimension error	create_collection 의 size 값이 다름	size=1024 (BGE-M3 기준) 인지 확인
대시보드에 Points 가 0개	upsert 실행 전에 대시보드를 확인함	upsert 코드가 에러 없이 끝났는지 먼저 확인 후 새로고침

필터 검색 — 서가를 좁혀서 찾기

개념

도서관에서 '전체 서가'가 아니라 '**2024년** 신간 코너'에서만 찾는 것

- 메타데이터(태그)를 저장해뒀기 때문에, 검색할 때 조건을 걸 수 있습니다.
- 예를 들어 '5페이지 이후 내용에서만 찾아줘' 같은 조건을 추가할 수 있습니다.
- 이게 바로 FAISS로는 못 하고 Qdrant로는 되는 대표적인 기능입니다.

Step 7~8. 검색·필터 — 문제 해결

이런 증상이 보이면	원인	해결 방법
필터 결과가 0개	저장된 page 값의 타입이 다름 (문자열 vs 숫자)	payload 저장 시 int(page) 로 숫자형인지 확인
ImportError: Filter	import 경로가 다름	from qdrant_client.models import Filter 확인
FAISS 와 Qdrant 결과가 조금 다름	정상입니다 — 근사 알고리즘 특성	완전히 같을 필요 없음, 상위 결과가 겹치는지만 확인

실습 1 완료 체크 + 보너스 실습

완료 체크리스트

- **PDF** 로딩 → 청킹 → 임베딩(**N, 1024**) 완료
- **FAISS** 인덱스 구축 및 검색 5개 확인
- **Qdrant** 컬렉션 생성 + 업로드 + 필터 검색 동작 확인

보너스: 청크 크기를 바꿔서 결과 비교, Qdrant 대시보드 직접 탐색해보기

실습에서 사용할 질문

기본 질문 (Easy → Hard)

- **HBM**이란 무엇인가? (Easy — 한 문서에서 바로 확인 가능)
- **AI** 반도체 시장을 견인하는 요인은? (Medium)
- 한국의 반도체 경쟁력은? (Medium~Hard — 여러 문서 참조 필요)

통합 질문(Hard):

"보고서들을 종합해 생성형 AI가 요구하는 핵심 반도체 기술과,
한국이 우선 강화해야 할 분야를 설명하시오"

이 질문에 답하는 것이 오늘의 최종 목표입니다

4) AI 서비스의 고도화 생성형 AI 서비스 개발 – Vector DB

8. 실습 – 검색을 더 똑똑하게

- Dense 검색의 약점을 직접 겪어보고 Hybrid로 개선합니다

실습 2 전체 흐름 미리보기

5단계로 진행합니다

- ① **BM25** 인덱스 — 옛날 방식(키워드) 검색 준비하기
- ② **Dense** 실패 케이스 — 의미 검색이 안 통하는 순간을 직접 확인
- ③ **RRF** 결합 — 키워드 검색과 의미 검색을 하나로 합치기
- ④ **Reranker** — 후보를 한 번 더 정밀하게 채점하기
- ⑤ **Before/After** 비교 + **RAGAS** — 얼마나 좋아졌는지 숫자로 확인

실습 2 흐름 — Hybrid Search + Re-ranking

1 1. BM25 인덱스 구축

bm25s 라이브러리

2 2. Dense 실패 케이스 확인

키워드 쿼리 테스트

3 3. RRF 결합 함수 구현

Dense + BM25 합산

4 4. BGE-Reranker 적용

Cross-encoder 재평가

5 5. Before/After 비교 + RAGAS 검증

세 단계 결과 비교

BM25 — 40년 넘게 쓰인 키워드 검색

개념

의미는 몰라도, '그 단어가 몇 번 나왔는지'는 정확하게 셀 수 있다

- BGE-M3 임베딩은 '의미'를 이해하지만, 'HBM' 같은 고유명사·약어는 놓치기도 합니다.
- BM25는 반대로 의미는 몰라도, 찾는 단어가 문서에 얼마나 자주·희귀하게 등장하는지를 계산해 점수를 매깁니다.
- 이번 단계에서는 이 BM25 검색 엔진을 준비합니다.

Step 1. BM25 — 문제 해결

이런 증상이 보이면	원인	해결 방법
결과가 하나도 없음	corpus와 query의 tokenize 방식이 다름	둘 다 <code>bm25s.tokenize()</code> 로 동일하게 처리했는지 확인
한국어 검색이 부정확함	띄어쓰기 기준이라 조사(은/는/이/가)가 섞임	오늘은 기본 토크나이저로 진행, 심화는 <code>konlpy</code> 참고(부록)

Dense가 실패하는 순간을 직접 목격하기

개념

"HBM"이라는 단어 자체보다, **'비슷한 분위기의 문장'**을 찾는 게 **Dense** 검색의 특징

- BGE-M3 임베딩(Dense)은 '의미가 비슷한 문장'을 찾는 데는 강하지만,
- 'HBM'처럼 고유한 약어 하나가 핵심인 질문에서는 엉뚱하게 '반도체 산업 개요' 같은 일반적인 문서를 가져오기도 합니다.
- 지금 두 검색을 나란히 실행해서 이 차이를 직접 확인해봅시다.

Dense 실패 케이스 직접 확인

❌ Dense 결과 (예상)

- 반도체 산업 개요
- AI 시장 동향
- 메모리 기술 소개
- 산업 전망 요약

BM25 결과 (예상)

- HBM 기술 상세 설명
- HBM 시장 점유율
- 차세대 HBM 로드맵

쿼리: "HBM" (고유명사·약어) — 직접 실행해서 두 결과를 비교해보세요

RRF — 두 검색 결과를 안전하게 합치는 법

개념

두 심사위원의 '점수'가 아니라 '등수'만 보고 합산한다

- Dense 점수(0~1)와 BM25 점수(0~수십)는 기준이 완전히 달라서 그냥 더하면 안 됩니다.
- 그래서 점수 대신 '몇 등이었는지(순위)'만 이용해서 안전하게 합산하는 방법이 RRF입니다.
- 두 검색 모두에서 상위권이었던 문서일수록 최종 점수가 높아집니다.

Step 3. RRF — 문제 해결

이런 증상이 보이면	원인	해결 방법
KeyError 또는 타입 에러	numpy int 와 python int 가 섞임	int(i) 로 명시적으로 형변환했는지 확인
Dense 후보와 BM25 후보 가 안 겹침	정상입니다 — 그래서 Hybrid 가 필요한 것	겹치지 않아도 RRF 는 두 후보를 모두 고려하니 걱정 없음
특정 문서가 항상 1등 으로 나옴	두 검색 모두에서 상위였다는 뜻(정상)	이상하면 rrf_k(60) 나 k*3 범위를 조정해서 테스트

Reranker — 서류 전형 다음, 대면 면접

개념

빠르게 뽑은 후보를, 이번엔 꼼꼼하게 한 명씩 다시 읽어보고 채점한다

- RRF로 뽑은 후보들은 '빠르지만 대략적인' 방식으로 골라진 것들입니다.
- Reranker는 질문과 각 후보 문서를 '한 쌍씩 직접' 비교하며 훨씬 정밀하게 점수를 매깁니다.
- 전체 문서가 아니라 후보 20~50개만 다시 보기 때문에 시간이 오래 걸리지 않습니다.

Step 4. Reranker — 문제 해결

이런 증상이 보이면	원인	해결 방법
모델 다운로드가 오래 걸림	최초 실행 시 1회만 발생하는 다운로드	몇 분 정도 소요될 수 있음, 인터넷 연결만 확인하고 대기
메모리 부족(OOM) 에러	reranker도 모델이라 메모리를 사용함	<code>use_fp16=True</code> 유지, <code>candidate_ids</code> 의 k값을 20 → 10으로 축소
<code>pairs</code> 형식 에러	리스트 안에 [질문,문서] 형태가 아님	<code>pairs = [[query, c] for c in candidates]</code> 형태인지 확인

세 단계 비교란 무엇을 확인하는 걸까?

개념

검색 방법을 한 단계씩 추가할 때마다, 결과가 실제로 좋아지는 걸 눈으로 확인하는 것

- 오늘 우리는 같은 질문(예: 'HBM 반도체')을 세 가지 다른 방식으로 검색해봤습니다.
 - ① Dense만 — Step 4까지 만든 BGE-M3 임베딩 검색 '단독' 결과
 - ② Hybrid(RRF) — 여기에 Step 1의 BM25(키워드 검색)까지 합친 결과
 - ③ +Reranker — Hybrid 결과를 Step 4에서 만든 Reranker로 한 번 더 정밀하게 재채점한 결과

세 단계 한눈에 정리

단계	이번 실습에서 실행한 함수	기대하는 변화
① Dense만	<code>index.search(q_vec, k=5)</code>	의미는 비슷하지만 'HBM'과 무관한 문서도 섞여 나옴
② Hybrid (RRF)	<code>hybrid_search_rrf(query, k=10)</code>	BM25 덕분에 'HBM' 키워드가 포함된 문서가 상위로 올라옴
③ +Reranker	<code>reranker.compute_score(pairs)</code>	후보들의 순서가 질문과의 관련도 기준으로 더 정밀해짐

이 세 함수를 같은 질문으로 각각 실행해서, 아래 표처럼 결과를 직접 비교합니다

Before / After — 세 단계 결과 비교 (예시)

순위	Dense만	Hybrid (RRF)	+Reranker
1위	반도체 산업 개요	HBM 기술 설명	HBM 기술 설명 ✓
2위	AI 시장 동향	HBM 시장 점유율	HBM 시장 점유율 ✓
3위	메모리 기술 소개	차세대 HBM 로드맵	차세대 HBM 로드맵 ✓

쿼리: "HBM 반도체" — 각 단계에서 순위가 실제로 어떻게 바뀌는지 직접 출력해서 확인해보세요

단계마다 무엇이 왜 좋아졌을까?

- 1** ① → ② **Dense만** → **Hybrid**
BM25가 'HBM'이라는 단어 자체를 직접 찾아주면서,
의미 검색이 놓친 키워드 매칭을 보완합니다
- 2** ② → ③ **Hybrid** → **+Reranker**
RRF로 뽑은 후보를 질문과 1:1로 비교해 재채점하면서
순위가 더 정밀해집니다
- 3** 리포트에 쓸 내용
세 단계 각각의 실행 결과(캡처 또는 출력 텍스트)를 담고,
'왜 개선됐는지' 한 줄씩 적으면 됩니다

이 세 단계 비교가 바로 제출 리포트에 들어갈 핵심 내용입니다

RAGAS — 감이 아니라 숫자로 확인하기

개념

"좋아진 것 같다"가 아니라 "**Faithfulness 0.6 → 0.9로 올랐다**"라고 말하기 위한 도구

- 지금까지는 결과를 눈으로 보고 '나아진 것 같다'고 판단했습니다.
- RAGAS는 질문·답변·검색된 문서만 넣으면 자동으로 여러 지표를 계산해줍니다.
- Faithfulness(근거에 충실한가), Answer Relevance(질문에 맞게 답했는가) 등을 점수로 확인

RAGAS로 결과 검증하기

RAG 전용 자동 평가 프레임워크

- **Faithfulness** — 답변이 검색된 문서 내용에 실제로 근거하는가?
- **Answer Relevance** — 답변이 질문의 의도에 맞게 대답하고 있는가?
- **Context Precision** — 검색된 문서 중 실제로 유용한 것의 비율은?

pip install ragas 로 설치, 질문-답변-검색결과만 있으면 바로 점수를 계산할 수 있습니다

Step 5. RAGAS — 문제 해결

이런 증상이 보이면	원인	해결 방법
OpenAI API 에러	RAGAS 내부적으로도 LLM을 호출해 채점함	OPENAI_API_KEY가 같은 터미널 환경에 설정돼 있는지 확인
점수가 낮게 나옴	정상적인 진단 결과일 수 있음	낮은 항목을 보고 청크 크기·검색 방식 중 무엇을 바꿀지 판단
실행이 느림	평가할 때마다 LLM을 호출하기 때문	질문 1~2개로 먼저 테스트 후 전체로 확장

실습 2 완료 체크 + 자유 실습

완료 체크리스트

- ☐ **BM25 인덱스 + Dense 실패 케이스 확인**
- ☐ **RRF 결합 함수 동작 확인 (Hybrid가 Dense보다 나은 것 확인)**
- ☐ **BGE-Reranker 적용 후 순위 변화 확인**
- ☐ **완전한 파이프라인으로 통합 질문 실행**

자유 실습: 다른 쿼리로 비교, RAGAS 점수 직접 계산, Parent-Child 구조 추가 실험

오늘 실습으로 직접 확인한 것들



임베딩 생성 시간

수백 개 청크에 수십 초
→ 10만개면 GPU가 필요한 이유



FAISS의 한계

메타데이터 저장 불가
→ 전용 서버(Qdrant)가 필요한 이유



Dense 실패 케이스

HBM 같은 고유명사 쿼리에서 직접 확인



RRF 결합 효과

키워드 관련 문서가 상위로 올라오는 것 확인



Reranker 순위 변화

같은 Top-5인데 순서가 달라지는 것 확인



RAGAS 검증

감으로만 판단하지 않고 숫자로 확인하는 습관

실습 제출물 안내

제출 대상 – 아래 3가지를 하나로 묶어도 무방 (캠퍼스_반_이름.zip으로 제출)

- ① 개발 코드 — 폴더 구조는 포함, 불필요한 폴더 제거
- ② 리포트 — 실습1 검색·필터 결과, 실습2 세 단계 비교 + **RAGAS** 검증 결과
+ **RAG** 설정 시 주요 고려 사항 정리 (개인 의견 중심)

자유 실습: 다른 쿼리로 비교, **RAGAS** 점수 직접 계산, **Parent-Child** 구조 추가 실험

[참고] 패키지 설치 및 환경 확인

패키지 설치

```
pip install pymupdf langchain faiss-cpu
```

```
pip install qdrant-client FlagEmbedding bm25s ragas
```

환경 확인

```
python -c "import fitz, faiss, qdrant_client; print('OK')"
```

```
from qdrant_client import QdrantClient
```

```
client = QdrantClient('localhost', port=6333)
```

```
print(client.get_collections())
```

```
# CollectionsResponse(collections=[]) 나오면 OK
```

OK 와 CollectionsResponse(collections=[]) 가 출력되면 준비 완료

[참고] Before/After 세 단계 비교

순위	Dense만	Hybrid(RRF)	+Reranker
1위	반도체 산업 개요	HBM 기술설명	HBM 기술 설명
2위	AI 시장 동향	HBM 시장 점유율	HBM 시장 점유율
3위	메모리 기술 소개	차세대 HBM 로드맵	차세대 HBM 로드맵

쿼리: "HBM 반도체" — 각 단계에서 순위가 어떻게 달라지는지 직접 확인!

[참고] Step 1. PDF 로딩 (PyMuPDF)

```
import fitz # PyMuPDF
def load_pdf(path):
    doc = fitz.open(path)
    pages = []
    for i, page in enumerate(doc):
        text = page.get_text()
        if text.strip():
            pages.append({'text': text, 'page': i+1})
    return pages

pages = load_pdf('spri_ai_brief.pdf')
print(f'총 {len(pages)}페이지 로딩 완료')
```

페이지 수와 텍스트가 출력되면 성공

[참고] Step 3. 임베딩 생성 (BGE-M3)

```
from FlagEmbedding import BGEM3FlagModel
import numpy as np

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

texts = [c['text'] for c in chunks]
output = model.encode(texts, batch_size=16,
                      return_dense=True)
embeddings = np.array(
    output['dense_vecs']).astype('float32')
print(f'shape: {embeddings.shape}') # (N, 1024)
```

shape가 (N, 1024)이면 성공

[참고] Step 2. 청킹 (RecursiveCharacterTextSplitter)

```
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50,
    separators=['\n\n', '\n', '.', ' '])

chunks = []
for page in pages:
    for split in splitter.split_text(page['text']):
        chunks.append({'text': split, 'page': page['page']})
print(f'총 청크 수: {len(chunks)}')
```

청크가 500자 전후로 여러 개 생성되면 성공

[참고] Step 4. FAISS 인덱스 구축 및 검색

```
import faiss

index = faiss.IndexHNSWFlat(1024, 32)
index.hnsw.efConstruction = 200
index.add(embeddings)

# 검색 테스트
query = 'HBM이란 무엇인가?'
q_vec = model.encode([query])['dense_vecs'].astype('float32')
dists, idxs = index.search(q_vec, k=5)
for i, idx in enumerate(idxs[0]):
    print(f'{i+1}위: {chunks[idx]["text"][:80]}')
```

5개 검색 결과가 출력되면 성공

[참고] Step 5~6. Qdrant Collection 생성 + Upsert

```
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance, PointStruct)

client = QdrantClient('localhost', port=6333)
client.create_collection(
    collection_name='rag_docs',
    vectors_config=VectorParams(
        size=1024, distance=Distance.COSINE))

points = [PointStruct(id=i, vector=embeddings[i].tolist(),
    payload={'text': chunks[i]['text'], 'page': chunks[i]['page']})
    for i in range(len(chunks))]
client.upsert('rag_docs', points)
```

대시보드(**localhost:6333/dashboard**)에서 포인트 수를 확인하면 성공

[참고] Step 7~8. 검색 비교 + 메타데이터 필터 검색

```
# Qdrant 검색 (score 함께 반환)
results = client.search(
    collection_name='rag_docs',
    query_vector=q_vec[0].tolist(), limit=5)

# 메타데이터 필터: 5페이지 이후만 검색
from qdrant_client.models import Filter, FieldCondition, Range
filtered = client.search(
    collection_name='rag_docs',
    query_vector=q_vec[0].tolist(),
    query_filter=Filter(must=[
        FieldCondition(key='page', range=Range(gte=5))]),
    limit=5)
```

Qdrant가 유사도 score를 반환하고, 필터 검색은 5페이지 이후만 나오면 성공

[참고] Step 1. BM25 인덱스 구축

```
import bm25s

texts = [c['text'] for c in chunks]
corpus_tokens = bm25s.tokenize(texts)
bm25_index = bm25s.BM25()
bm25_index.index(corpus_tokens)

query = 'HBM 반도체'
q_tok = bm25s.tokenize(query)
results, scores = bm25_index.retrieve(q_tok, k=5)
```

[참고] Step 3. RRF 결합 함수 구현

```
def hybrid_search_rrf(query, k=10, rrf_k=60):  
    q_vec = model.encode([query])['dense_vecs'].astype('float32')  
    _, d_ids = index.search(q_vec, k*3)  
    dense_ranks = {int(i): r+1 for r, i in enumerate(d_ids[0])}  
  
    q_tok = bm25s.tokenize(query)  
    s_ids, _ = bm25_index.retrieve(q_tok, k=k*3)  
    sparse_ranks = {int(i): r+1 for r, i in enumerate(s_ids[0])}  
  
    all_ids = set(dense_ranks) | set(sparse_ranks)  
    rrf = {d: 1/(rrf_k+dense_ranks.get(d,9999))  
           + 1/(rrf_k+sparse_ranks.get(d,9999)) for d in all_ids}  
    return sorted(rrf, key=rrf.get, reverse=True)[:k]
```

[참고] Step 4. BGE-Reranker 적용

```
from FlagEmbedding import FlagReranker

reranker = FlagReranker(
    'BAAI/bge-reranker-v2-m3', use_fp16=True)

candidate_ids = hybrid_search_rrf(query, k=20)
candidates = [chunks[i]['text'] for i in candidate_ids]
pairs = [[query, c] for c in candidates]

scores = reranker.compute_score(pairs, normalize=True)
reranked = sorted(zip(scores, candidates), reverse=True)
```



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.